

Introduction to SRE

Midterm

Anton Shaubert, SE-2430

Presentation: <https://canva.link/xmij8281765c8s2>

STEP 1:

1. Introduction and Objective

This project bridges Software Engineering and Site Reliability Engineering (SRE) by transforming my existing full-stack web application into a production-style system with containerization, reliability targets, monitoring dashboards, and alerting.

I used my own application (not a template):

- **Frontend:** React (Vite), served by Nginx
- **Backend:** Node.js + Express API
- **Database:** MongoDB

The main SRE goal was to define measurable system health (SLIs/SLOs), continuously observe it with Prometheus + Grafana, and validate alerting behavior.

2. System Architecture

The deployed stack contains:

1. **Frontend service** (frontend)
2. **Backend API service** (backend)
3. **Database service** (mongo)
4. **Observability stack**
 - a. prometheus (metrics collection + alert rules)
 - b. grafana (dashboards/visualization)
 - c. node-exporter (host/container resource metrics)

Key files

- frontend/albumlog-front-main/Dockerfile
- frontend/albumlog-front-main/nginx.conf
- backend/Dockerfile

- docker-compose.yml
- docker-stack.yml (bonus Swarm)
- Prometheus/prometheus.yml
- prometheus/alert_rules.yml

Containerization (Docker Compose)

What was implemented

- Created Dockerfile for frontend with multi-stage build (Node build -> Nginx runtime).
- Created Dockerfile for backend (Node runtime with production dependencies).
- Created compose stack that runs frontend, backend, MongoDB, Prometheus, Grafana, and node-exporter.
- Added .env-based configuration for clean environment management.

Docker-compose.yml

```
services:
  frontend:
    build:
      context: .\frontend\albumlog-front-main
      args:
        VITE_API_BASE_URL: ${VITE_API_BASE_URL}
    ports:
      - "5173:80"
    depends_on:
      - backend

  backend:
    build:
      context: .\backend
    env_file:
      - .\backend\.env
    ports:
      - "4444:4444"
    depends_on:
      - mongo

  prometheus:
    image: prom/prometheus:latest
```

```

command:
  - "--config.file=/etc/prometheus/prometheus.yml"
  - "--storage.tsdb.path=/prometheus"
  - "--web.enable-lifecycle"
ports:
  - "9090:9090"
volumes:
  - .\prometheus\prometheus.yml:/etc/prometheus/prometheus.yml:ro
  - .\prometheus\alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
depends_on:
  - backend
  - node-exporter

grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  env_file:
    - .\.env
  volumes:
    - grafana_data:/var/lib/grafana
  depends_on:
    - prometheus

node-exporter:
  image: prom/node-exporter:latest
  ports:
    - "9100:9100"

volumes:
  mongo_data:
  grafana_data:

```

Prometheus.yml:

```

global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - /etc/prometheus/alert_rules.yml

scrape_configs:
  - job_name: prometheus
    static_configs:

```

```
- targets: ["prometheus:9090"]

- job_name: backend
  metrics_path: /metrics
  static_configs:
    - targets: ["backend:4444"]

- job_name: node_exporter
  static_configs:
    - targets: ["node-exporter:9100"]
```

Backend Dockerfile:

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --omit=dev

COPY . .

EXPOSE 4444

CMD ["npm", "start"]
```

Frontend Dockerfile:

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
ARG VITE_API_BASE_URL=http://localhost:4444
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
PS C:\Users\pc2\Downloads\sre> docker compose up -d
[+] up 7/7
✓ Network sre_default Created 0.0s
✓ Container sre-node-exporter-1 Started 1.4s
✓ Container sre-mongo-1 Started 1.4s
✓ Container sre-backend-1 Started 1.5s
✓ Container sre-frontend-1 Started 1.6s
✓ Container sre-prometheus-1 Started 1.7s
✓ Container sre-grafana-1 Started 1.7s
PS C:\Users\pc2\Downloads\sre>
```

Prometheus

Query

Alerts

Status > Target health

Select scrape pool

Filter by target health

Filter by endpoint or labels

backend

1 / 1 up

Endpoint

Labels

Last scrape

State

http://backend:4444/metrics

instance="backend:4444" job="backend"

11.718s ago

UP

node_exporter

1 / 1 up

Endpoint

Labels

Last scrape

State

http://node-exporter:9100/metrics

instance="node-exporter:9100" job="node_exporter"

7.994s ago

UP

prometheus

1 / 1 up

Endpoint

Labels

Last scrape

State

http://prometheus:9090/metrics

instance="prometheus:9090" job="prometheus"

679ms ago

UP

Find album or artist...

My profile

Top rock albums

Rock Pop Indie Rap Hip-Hop

Born in the U.S.A.

Bruce Springsteen

See album

A Rush of Blood to the Head

Coldplay

See album

Stadium Arcadium

Red Hot Chili Peppers

See album

14 Shades of Grey

Recent Reviews

test

Stadium Arcadium

Red Hot Chili Peppers

Too Much of a Good Thing

Stadium Arcadium by Red Hot Chili Peppers is ambitious to a fault—a double album packed with ideas, moods, and styles that don't always justify their length. At its best, it's fantastic. Tracks like Dani California, Snow (Hey Oh), and Wet Sand capture the band at a creative peak, with John Frusciante's guitar work standing out as the emotional core. There's a polished, almost dreamy quality to a lot of the production that works really well. But across 28 tracks, the album struggles with excess. There's clear filler, and the pacing drags, making it hard to experience as a full listen rather than picking highlights. It lacks the tight focus of something like Californication. There's easily a great single album hidden in here—but as it stands, it's overextended. Strong moments, inconsistent whole.

4 / 5

test

1 (Remastered)

The Beatles

Greatest Hits, Not a Greatest Album

1 by The Beatles is almost impossible to criticize in terms of song quality—and yet, it's hard to

STEP 2:

SLI 1: API Availability

Definition: Ratio of successful backend requests to total backend requests.

PromQL:

```
sum(rate(http_requests_total{status_code!~"5.."}[5m])) /  
sum(rate(http_requests_total[5m]))
```

SLO 1: Monthly availability $\geq 99.5\%$

Error Budget Calculation (30-day month)

- Total minutes/month = $30 \times 24 \times 60 = 43,200$
- Allowed failure = 0.5%
- Error budget = $43,200 \times 0.005 = 216$ minutes/month

So the service can be outside target for up to **216 minutes/month**.

SLI 2: API Latency (p95)

Definition: 95th percentile backend response time.

PromQL:

```
histogram_quantile(0.95, sum by (le)  
(rate(http_request_duration_seconds_bucket[5m])))
```

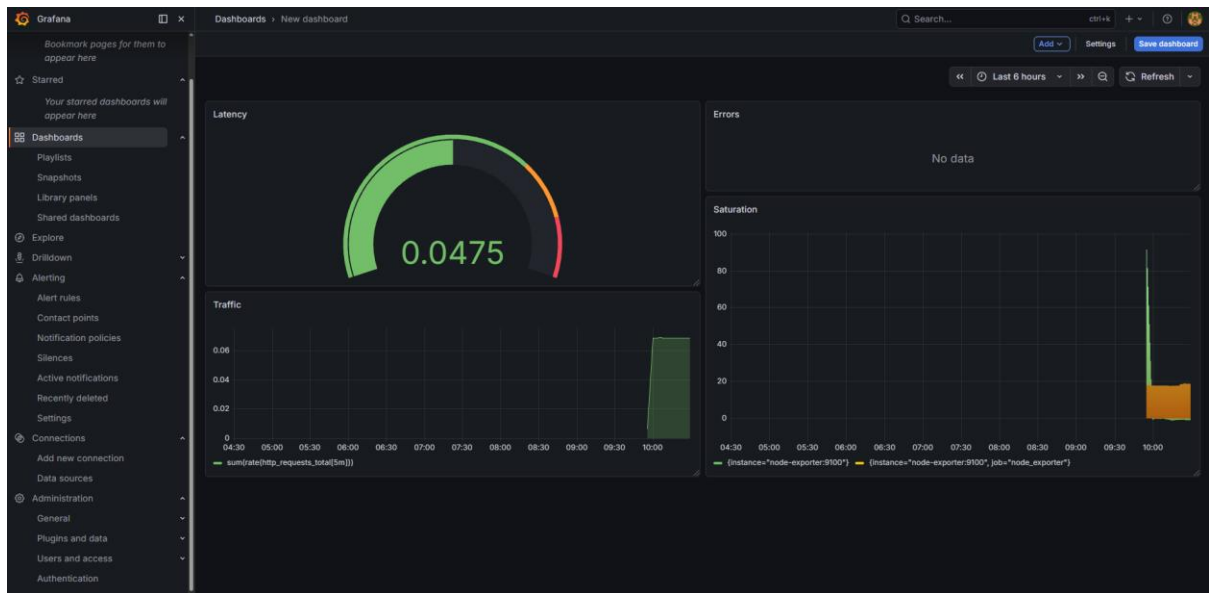
SLO 2: p95 latency < 800 ms in at least **95% of 5-minute windows** in a month.

Error Budget Interpretation for Latency

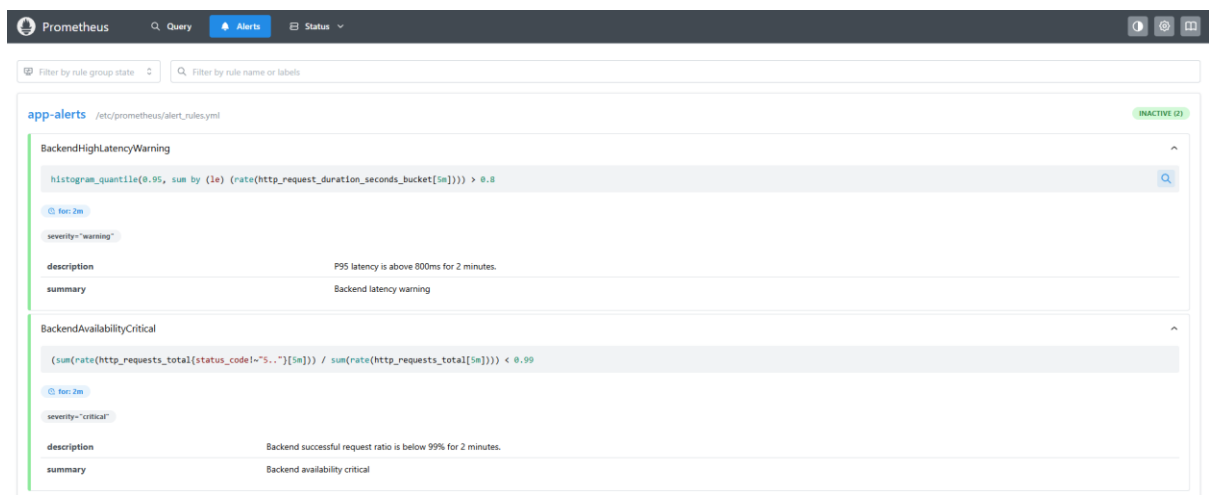
- 5-minute windows/month = $43,200 / 5 = 8,640$
- Allowed violation windows = $8,640 \times 0.05 = 432$

So latency target may be violated in at most **432 windows/month**.

STEP 3:



STEP 4:



Alert-rules.yml:

```
groups:
- name: app-alerts
  rules:
  - alert: BackendHighLatencyWarning
    expr: histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m]))) > 0.8
    for: 2m
    severity: warning
    description: P95 latency is above 800ms for 2 minutes.
    summary: Backend latency warning
  - alert: BackendAvailabilityCritical
    expr: (sum(rate(http_requests_total(status_code=~"5..")[5m])) / sum(rate(http_requests_total[5m]))) < 0.99
    for: 2m
    severity: critical
    description: Backend successful request ratio is below 99% for 2 minutes.
    summary: Backend availability critical
```

```

description: "P95 latency is above 800ms for 2 minutes."

- alert: BackendAvailabilityCritical
  expr: (sum(rate(http_requests_total{status_code!~"5.."}[5m])) /
sum(rate(http_requests_total[5m]))) < 0.99
  for: 2m
  labels:
    severity: critical
  annotations:
    summary: "Backend availability critical"
    description: "Backend successful request ratio is below 99% for 2
minutes."

```

The screenshot shows the Prometheus Alerts interface. At the top, there's a navigation bar with 'Prometheus', 'Query', 'Alerts' (active), and 'Status'. Below the navigation bar, there are filters for 'Filter by rule group state' and 'Filter by rule name or labels'. The main content area displays two alerts from the 'app-alerts' rule group.

BackendHighLatencyWarning (FIRING (1))

- Expression: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])) > 0.8`
- For: 2m
- Severity: warning
- Description: P95 latency is above 800ms for 2 minutes.
- Summary: Backend latency warning

BackendAvailabilityCritical (FIRING (1))

- Expression: `(sum(rate(http_requests_total{status_code!~"5.."}[5m])) / sum(rate(http_requests_total[5m]))) < 0.99`
- For: 2m
- Severity: critical
- Description: Backend successful request ratio is below 99% for 2 minutes.
- Summary: Backend availability critical

At the bottom, there is a table for 'Alert labels' with columns for 'Alert labels', 'State', 'Active Since', and 'Value'.

Alert labels	State	Active Since	Value
alertname="BackendAvailabilityCritical" severity="critical"	FIRING	2m 15.119s	0.0023589555565131553

Using command:

```
while ($true) { curl.exe -s "http://localhost:4444/reviews/invalid-id" | Out-Null }
```

I simulated firing an alert, which works.

BONUS: Docker swarm


```

PS C:\Users\pc2\Downloads\sre> docker stack services sre
>> docker stack ps sre
ID                NAME                MODE                REPLICAS    IMAGE                PORTS
g5m5vctlp9gq     sre_backend         replicated          2/2          albumlog-backend:latest *:4444->4444/tcp
1nw8zcyr0n2m     sre_frontend        replicated          2/2          albumlog-frontend:latest *:5173->80/tcp
gkyit7q40d1t     sre_grafana         replicated          1/1          grafana/grafana:latest  *:3000->3000/tcp
rer9ewydlpp9     sre_mongo           replicated          1/1          mongo:7
t87mmiyotgck     sre_node-exporter   global             1/1          prom/node-exporter:latest *:9100->9100/tcp
3rmin8amvv4qc     sre_prometheus      replicated          1/1          prom/prometheus:latest  *:9090->9090/tcp

PS C:\Users\pc2\Downloads\sre> docker build -t albumlog-frontend:latest .\frontend\albumlog-front-main
>> docker build -t albumlog-backend:latest .\backend
[+] Building 93.1s (15/15) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 417B                                0.0s
=> [internal] load metadata for docker.io/library/nginx:1.27-alpine 1.6s
=> [internal] load metadata for docker.io/library/node:20-alpine    3.4s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 81B                                      0.0s
=> [build 1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb 14.0s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f60 0.0s
=> => sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 1.26MB / 1.26MB 1.3s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 445B / 445B 1.1s
=> => sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 43.23MB / 43.23MB 11.2s
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB 3.9s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 0.2s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3db96603b1c009c797cfdde77bea8b3287 2.5s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892edcc4ce74c5a48d82e1d2b382edac3122 0.1s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb288d6f2b6a07a8bb44c60e8c075d2096 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 3.83kB                                    0.0s
=> [stage-1 1/3] FROM docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a 0.0s
=> => resolve docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76 0.0s
=> [build 2/6] WORKDIR /app                                         0.2s
=> [build 3/6] COPY package*.json ./                                0.1s

```

```

PS C:\Users\pc2\Downloads\sre> docker stack deploy -c docker-stack.yml sre
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service sre_node-exporter (id: t87mmiyotgckoaqto33tnjzvc)
Updating service sre_frontend (id: 1nw8zcyr0n2m7yls8sjzy8wav)
image albumlog-frontend:latest could not be accessed on a registry to record
its digest. Each node will access albumlog-frontend:latest independently,
possibly leading to different nodes running different
versions of the image.

Updating service sre_backend (id: g5m5vctlp9gq81owpul3jhs1x)
image albumlog-backend:latest could not be accessed on a registry to record
its digest. Each node will access albumlog-backend:latest independently,
possibly leading to different nodes running different
versions of the image.

Updating service sre_prometheus (id: 3rmn8amvv4qc0hh6bci9xv670)
Updating service sre_grafana (id: gkyit7q40d1tv2clfobnplmws)

```

Running 2 minutes ago

```

PS C:\Users\pc2\Downloads\sre> docker service scale sre_backend=3
>>
sre_backend scaled to 3
overall progress: 3 out of 3 tasks
1/3: running [=====>]
2/3: running [=====>]
3/3: running [=====>]
verify: Service sre_backend converged
PS C:\Users\pc2\Downloads\sre> docker service ls

```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
g5m5vctlp9gq	sre_backend	replicated	3/3	albumlog-backend:latest	*:4444->4444/tcp
1nw8zcyr0n2m	sre_frontend	replicated	2/2	albumlog-frontend:latest	*:5173->80/tcp
gkyit7q40d1t	sre_grafana	replicated	1/1	grafana/grafana:latest	*:3000->3000/tcp
rer9ewydlpp9	sre_mongo	replicated	1/1	mongo:7	
t87mmiyotgck	sre_node-exporter	global	1/1	prom/node-exporter:latest	*:9100->9100/tcp
3rmn8amvv4qc	sre_prometheus	replicated	1/1	prom/prometheus:latest	*:9090->9090/tcp
4aldgjd6fpx7	web	replicated	5/5	nginx:1.27	*:8080->80/tcp

```

PS C:\Users\pc2\Downloads\sre>

```